

*TUTORIAL 3- BACKPROPAGATION AND MODEL TRAINING *

```

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms

# Define a simple MLP model
class MLP(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_size, hidden_size)
        self.fc3 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x)
        return x

input_size = 32 * 32 * 3 # CIFAR-10 images are 32x32 pixels with 3 channels (RGB)
hidden_size = 256
num_classes = 10
learning_rate = 0.001
num_epochs = 10
batch_size = 100

# Load CIFAR-10 dataset
transform = transforms.Compose([
    transforms.ToTensor(), # Convert to Tensor
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)) # Normalize with mean and std deviation for each channel
])

train_dataset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size, shuffle=True)

Downloading https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz to ./data/cifar-10-python.tar.gz
100%|██████████| 170498071/170498071 [00:03<00:00, 43451528.75it/s]
Extracting ./data/cifar-10-python.tar.gz to ./data

# Initialize the model, loss function, and optimizer
model = MLP(input_size, hidden_size, num_classes) # Initializing the MLP model

# Define the loss function
criterion = nn.CrossEntropyLoss() # Using Cross Entropy Loss for classification tasks

# Define the optimizer
optimizer = optim.Adam(model.parameters(), lr=learning_rate) # Adam optimizer with the given learning rate

```

```

# Training the model
for epoch in range(num_epochs):
    total_correct = 0
    total_samples = 0
    for i, (images, labels) in enumerate(train_loader):
        images = images.view(-1, 32 * 32 * 3) # Flatten the images from 32x32x3 to 1x(32*32*3)

        # Forward pass
        outputs = model(images)
        loss = criterion(outputs, labels)

        # Backward and optimize
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # Accuracy calculation
        _, predicted = torch.max(outputs, 1)
        total_samples += labels.size(0)
        total_correct += (predicted == labels).sum().item()

    # Print training statistics
    if (i+1) % 100 == 0:
        print(f'Epoch [{epoch+1}/{num_epochs}], Step [{i+1}/{len(train_loader)}], Loss: {loss.item():.4f}, Accuracy: {(total_correct/tot

print("Training finished")

# Save the trained model
torch.save(model.state_dict(), 'mlp_cifar10_model.pth')

```

```

Epoch [1/10], Step [100/500], Loss: 1.6176, Accuracy: 35.26%
Epoch [1/10], Step [200/500], Loss: 1.5499, Accuracy: 37.91%
Epoch [1/10], Step [300/500], Loss: 1.6131, Accuracy: 39.70%
Epoch [1/10], Step [400/500], Loss: 1.8132, Accuracy: 40.97%
Epoch [1/10], Step [500/500], Loss: 1.7093, Accuracy: 42.08%
Epoch [2/10], Step [100/500], Loss: 1.5786, Accuracy: 49.69%
Epoch [2/10], Step [200/500], Loss: 1.4602, Accuracy: 49.48%
Epoch [2/10], Step [300/500], Loss: 1.4894, Accuracy: 49.63%
Epoch [2/10], Step [400/500], Loss: 1.2494, Accuracy: 50.13%
Epoch [2/10], Step [500/500], Loss: 1.3593, Accuracy: 50.24%
Epoch [3/10], Step [100/500], Loss: 1.1659, Accuracy: 53.98%
Epoch [3/10], Step [200/500], Loss: 1.1494, Accuracy: 54.62%
Epoch [3/10], Step [300/500], Loss: 1.4811, Accuracy: 54.29%
Epoch [3/10], Step [400/500], Loss: 1.4690, Accuracy: 54.32%
Epoch [3/10], Step [500/500], Loss: 1.2730, Accuracy: 54.22%
Epoch [4/10], Step [100/500], Loss: 1.2265, Accuracy: 57.70%
Epoch [4/10], Step [200/500], Loss: 1.3183, Accuracy: 57.73%
Epoch [4/10], Step [300/500], Loss: 1.0724, Accuracy: 57.53%
Epoch [4/10], Step [400/500], Loss: 1.2538, Accuracy: 57.48%
Epoch [4/10], Step [500/500], Loss: 1.2215, Accuracy: 57.41%
Epoch [5/10], Step [100/500], Loss: 1.2905, Accuracy: 61.11%
Epoch [5/10], Step [200/500], Loss: 1.1174, Accuracy: 60.50%
Epoch [5/10], Step [300/500], Loss: 1.1487, Accuracy: 60.02%
Epoch [5/10], Step [400/500], Loss: 1.3301, Accuracy: 59.79%
Epoch [5/10], Step [500/500], Loss: 1.2340, Accuracy: 59.85%
Epoch [6/10], Step [100/500], Loss: 0.9328, Accuracy: 64.20%
Epoch [6/10], Step [200/500], Loss: 1.0374, Accuracy: 63.56%
Epoch [6/10], Step [300/500], Loss: 1.1103, Accuracy: 63.10%
Epoch [6/10], Step [400/500], Loss: 1.0953, Accuracy: 62.98%
Epoch [6/10], Step [500/500], Loss: 1.1033, Accuracy: 62.90%
Epoch [7/10], Step [100/500], Loss: 0.9262, Accuracy: 66.55%
Epoch [7/10], Step [200/500], Loss: 0.8366, Accuracy: 66.42%
Epoch [7/10], Step [300/500], Loss: 0.8744, Accuracy: 66.07%
Epoch [7/10], Step [400/500], Loss: 0.9654, Accuracy: 65.68%
Epoch [7/10], Step [500/500], Loss: 1.0418, Accuracy: 65.43%
Epoch [8/10], Step [100/500], Loss: 0.7006, Accuracy: 69.71%
Epoch [8/10], Step [200/500], Loss: 0.8056, Accuracy: 69.16%
Epoch [8/10], Step [300/500], Loss: 0.9825, Accuracy: 68.61%
Epoch [8/10], Step [400/500], Loss: 0.7869, Accuracy: 68.21%
Epoch [8/10], Step [500/500], Loss: 0.9713, Accuracy: 67.74%
Epoch [9/10], Step [100/500], Loss: 0.7041, Accuracy: 70.89%
Epoch [9/10], Step [200/500], Loss: 0.7725, Accuracy: 70.53%
Epoch [9/10], Step [300/500], Loss: 0.8752, Accuracy: 70.46%
Epoch [9/10], Step [400/500], Loss: 0.8249, Accuracy: 70.10%
Epoch [9/10], Step [500/500], Loss: 0.9177, Accuracy: 69.84%
Epoch [10/10], Step [100/500], Loss: 0.6015, Accuracy: 74.02%
Epoch [10/10], Step [200/500], Loss: 1.0138, Accuracy: 73.87%
Epoch [10/10], Step [300/500], Loss: 0.8449, Accuracy: 73.29%
Epoch [10/10], Step [400/500], Loss: 0.6939, Accuracy: 72.69%

```

Epoch [10/10], Step [500/500], Loss: 0.8722, Accuracy: 72.30%
 Training finished

```
# Define the model architecture and load the saved model
model = MLP(input_size, hidden_size, num_classes) # Define the model with the same architecture
model.load_state_dict(torch.load('mlp_cifar10_model.pth')) # Load the saved model state

# Set the model to evaluation mode
model.eval()

# Load CIFAR-10 test dataset
test_dataset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

# Test the model on the test data
correct = 0
total = 0
with torch.no_grad():
    for images, labels in test_loader:
        images = images.view(-1, 32 * 32 * 3) # Flatten the images from 32x32x3 to 1x(32*32*3)
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

# Print the accuracy results
print(f'Accuracy on the test set: {(100 * correct / total):.2f}%')
```

Files already downloaded and verified
 Accuracy on the test set: 54.44%